

SendR - Project Documentation

Authors

Name	Student ID
Franciszek Przybyłowski	s223346
Jakub Orchowski	s223281
Kamil Pizon	s223434
Filip Obuchowicz	s223421

Contents

- [Introduction](#)
- [System Architecture](#)
- [Technology Stack](#)
- [Key Features](#)
- [Environment Configuration](#)
- [Development Process](#)

Introduction

Overview

SendR is a file-sharing web application inspired by services such as WeTransfer. The goal of the project is to provide a convenient platform for uploading, sharing, downloading, and managing files while keeping security requirements central to the design.

Key Design Decision The most significant decision made during the project was to **shift from a convenience-first approach to a security-first approach**. This meant introducing mandatory user authentication, even at the cost of the frictionless experience that services like WeTransfer are known for. A user now must authenticate via email before uploading or managing files.

This decision shaped the entire architecture of the application: authentication, tier-based access, quota enforcement, and file expiry are all consequences of prioritizing a secure environment over anonymous access.

Project Goals

The primary objectives of the project were:

- Build a functional file-sharing service with a modern web interface
- Implement a secure, passwordless authentication system based on email verification
- Enforce per-user upload limits and file expiry policies
- Package the application for containerized deployment using Docker
- Deploy the application to a cloud environment

System Architecture

High-Level Overview

SendR follows a classic client-server architecture, split into two independently deployable services:

- **Frontend** — an Angular single-page application served by an nginx web server
- **Backend** — a Python REST API built with FastAPI

The two services communicate exclusively through a well-defined HTTP API. The frontend never accesses the database directly. In local development, the Angular dev server proxies relative `/api/*` requests to the backend. Production frontend builds use the configured browser API origin from `environment.prod.ts`; the nginx container also keeps an `/api/*` reverse proxy configured by the runtime `API_URL` environment variable for same-origin/container deployments.

Architecture Summary Local dev: **Frontend** (Angular dev server) → **/api/* proxy** → **Backend** (FastAPI) → **Database** (SQLite)

Live: **Frontend** (Angular + nginx) -> **API origin / nginx proxy** -> **Backend** (FastAPI) -> **PostgreSQL + Spaces**

Frontend

The frontend is a single-page application (SPA) built with Angular 21. It is responsible for all user-facing interactions: uploading files, authenticating via email, browsing upload history, and managing file links.

A notable aspect of the frontend setup is the use of an **auto-generated API client**. The OpenAPI specification is exported from the running FastAPI backend, and the Angular client code is generated from it automatically. This ensures that the frontend and backend are always in sync with respect to request and response shapes, without requiring manual maintenance of API interfaces.

Backend

The backend is a RESTful API built with **FastAPI**, a modern Python web framework known for its performance and automatic documentation generation. The backend handles:

- User authentication (email-based, passwordless)
- File upload, storage, and retrieval
- Quota enforcement per user tier
- File expiry and link management
- Subscription tier management (Temporary, Free, Premium)

Database interactions are handled through **SQLModel**, which combines SQLAlchemy and Pydantic into a single model definition. Database schema changes are managed with **Alembic** migrations.

Database

Local development uses **SQLite** by default because it is simple and requires no external service. The live deployment uses **PostgreSQL** for relational data and **DigitalOcean Spaces** for upload payloads. SQLModel keeps model definitions portable across both database engines, while Alembic manages schema migrations.

Deployment Model

Both the frontend and backend are packaged as **Docker containers**. The live deployment runs on Kubernetes using the manifests under [k8s/overlays/live](#), with Terraform under [terraform/](#) managing the DigitalOcean cluster, database, Spaces bucket, VPC, and DNS records. The backend image is reused for both the API process and the scan worker process; ClamAV runs as a separate service.

Technology Stack

Frontend Technologies

Technology	Purpose
Angular 21	Main frontend framework (SPA)
TypeScript	Programming language for the frontend
SCSS	Styling
OpenAPI Generator	Auto-generates the HTTP client from the backend API spec
Bun	JavaScript runtime and package manager
nginx	Serves the frontend and proxies API requests

Backend Technologies

Technology	Purpose
Python	Programming language for the backend
FastAPI	Web framework for building the REST API
SQLModel	ORM combining SQLAlchemy and Pydantic
Alembic	Database migration management
SQLite	Local relational database
PostgreSQL	Live relational database
uv	Python package and environment manager

Tooling and Infrastructure

Tool	Purpose
Docker	Containerization of both services
Kubernetes	Live workload orchestration
Terraform	DigitalOcean infrastructure management
GitHub Actions	Continuous integration (CI) pipeline
pre-commit	Automated code quality checks before commit

Tool	Purpose
ruff	Python linter and formatter
oxlint	TypeScript/JavaScript linter
Renovate	Automated dependency updates

Key Features

Email-Based Authentication

SendR uses a **passwordless authentication** system. Instead of traditional username/password login, users provide their email address and receive a one-time verification code. This approach eliminates the risk of weak or reused passwords while keeping the login experience simple.

In production mode, verification codes are sent via SMTP. During local development, codes are printed directly to the server log to avoid the need for an email server setup.

File Sharing

The core feature of SendR is file upload and sharing. After authenticating, users can upload files which are then accessible via unique download links. The system supports file expiry, after which links become invalid and files are eventually purged.

CAPTCHA Protection

To prevent automated abuse of the authentication and upload endpoints, SendR integrates CAPTCHA verification. CAPTCHA is enabled by default in production mode (SENDR_ENVIRONMENT=production) and is relaxed in local development mode (SENDR_ENVIRONMENT=local) to streamline the development workflow.

Tier-Based Access Control

SendR implements a three-tier user system that controls upload limits and available features:

Feature	Temporary	Free	Premium
Email required	Yes	Yes	Yes
Files per week	3	5	50
Max file size	100 MB	1 GB	10 GB
Browse history	No	Yes	Yes
Edit files	No	No	Yes
Retrieve expired files	No	No	Yes

File Expiry and Refresh

Files uploaded to SendR are not stored indefinitely. Each file has a configurable expiry period. Users with a Premium tier can retrieve files even after expiry within a grace period, as well as generate new download links

for existing files.

Upload Quota Management

The system tracks upload usage per user on a rolling weekly basis. When a user reaches their tier's weekly upload limit, further transfers are rejected until quota resets. This ensures fair resource usage across all accounts.

API Documentation

Because the backend is built with FastAPI, interactive API documentation is automatically generated and available at runtime:

- **Swagger UI** — available at </docs>
- **ReDoc** — available at </redoc>

Environment Configuration

SendR uses environment variables to control runtime behaviour. The table below documents all supported variables.

- **SENDR_ENVIRONMENT** (default **production**): Set to **local** for local-only verification-code logging and relaxed CAPTCHA. Use **production** for real deployments.
- **SENDR_DEV_LOGIN_ENABLED** (default **false**): Exposes /api/dev/login/* only when this is **true** and **SENDR_ENVIRONMENT=local**. Never enable outside local dev.
- **SENDR_ALLOWED_ORIGINS**: Comma-separated list or JSON array of allowed CORS origins. Required when frontend and API are on different origins.
- SMTP settings: SMTP host, port, and credentials required for production email delivery. Not needed in local mode.
- **SENDR_DATABASE_URL**: SQLAlchemy-compatible database URL. Local development defaults to SQLite; live deployment uses PostgreSQL.
- **SENDR_SECRET_KEY**: Required outside local/test environments.
- **SENDR_VIRUS_SCANNING_ENABLED**: Must be **true** outside local/test environments.
- **SENDR_UPLOAD_DIR** and **SENDR_UPLOAD_QUARANTINE_DIR**: Local-disk clean and quarantine storage paths.
- **SENDR_SPACES_ACCESS_KEY**, **SENDR_SPACES_SECRET_KEY**, **SENDR_SPACES_BUCKET_NAME**, **SENDR_SPACES_REGION**: Required outside local/test environments for DigitalOcean Spaces object storage.
- **SENDR_ALTCHA_HMAC_KEY**: Required outside local/test environments so CAPTCHA challenges verify across backend replicas.
- Frontend production API origin: [frontend/src/environments/environment.prod.ts](#) configures the browser API origin. The nginx container also receives an upstream backend URL through **API_URL** for /api/* proxying.

Local development: In **SENDR_ENVIRONMENT=local**, verification codes are printed to the server log instead of being sent by email, and CAPTCHA verification is relaxed. Set **SENDR_DEV_LOGIN_ENABLED=true** only when you explicitly want dev-login shortcuts.

Development Process

Local Setup

Backend development starts from the `backend/` directory:

```
uv sync
uv run alembic upgrade head
SENDR_ENVIRONMENT=local SENDR_SECRET_KEY=local-dev-secret uv run uvicorn
src.app:app --host 0.0.0.0 --port 8000
```

Frontend development starts from the `frontend/` directory:

```
bun install
bun run start
```

The Angular dev server defaults to the `local-dev` serve configuration. It uses `frontend/proxy.conf.json` to forward `/api` to `http://localhost:8000`.

API Contract Workflow

FastAPI is the source of truth for the HTTP contract. The generated OpenAPI document lives at `openapi.json`, and the Angular client lives under `frontend/src/app/api`.

After changing backend endpoints, schemas, response models, or generated client inputs, run:

```
./scripts/generate-openapi-client.sh
```

Generated API files should be reviewed and committed with the backend change that required them.

Hooks And CI

Run the repository hook setup once per clone:

```
./setup-git-hooks.sh
```

The pre-commit hook can regenerate API client files and stages them into the same commit. It also runs frontend formatting, linting, and build checks for frontend-related commits. CI repeats the stale-client check by regenerating `openapi.json` and `frontend/src/app/api` and failing if a diff appears.

Verification Commands

Backend verification:

```
cd backend
uv run pytest
```

Frontend verification:

```
cd frontend
bun run format:check
bun run lint
bun run build --configuration production --base-href /SendR/
```

Frontend targeted tests:

```
cd frontend
bun run test -- --watch=false --browsers=ChromeHeadlessNoSandbox --progress=false
```

Authentication, refresh, logout, admin access, upload, and protected download flows should be manually checked after auth or access-control changes.

Deployment Workflow

Static Terraform validation and application validation run in CI. Real cloud changes are gated through the deployment workflow and the `SENDR_AUTO_DEPLOY_ENABLED=true` repository variable. The live deployment uses [k8s/overlays/live](#) and the Terraform configuration documented in [terraform/README.md](#), [terraform/SETUP.md](#), and [terraform/ENVIRONMENTS_GUIDE.md](#).

Data Migration

Cross-Database Migration Strategy

SendR now includes an offline migration CLI for moving business data between environments even when the database engine changes, for example from local SQLite to production PostgreSQL. The migration is based on a portable bundle instead of SQL dumps, because the application stores both relational data and upload payloads outside the database.

The CLI lives in the backend and should be run from the `backend/` directory:

```
uv run python src/migration_cli.py validate-source --database-url "$SOURCE_DB_URL"
--upload-dir "$SOURCE_UPLOAD_DIR" --quarantine-dir "$SOURCE_QUARANTINE_DIR"
uv run python src/migration_cli.py export-bundle --database-url "$SOURCE_DB_URL" -
--upload-dir "$SOURCE_UPLOAD_DIR" --quarantine-dir "$SOURCE_QUARANTINE_DIR" --
bundle-dir /tmp/sendr-bundle
uv run python src/migration_cli.py import-bundle --database-url "$TARGET_DB_URL" -
--upload-dir "$TARGET_UPLOAD_DIR" --quarantine-dir "$TARGET_QUARANTINE_DIR" --
bundle-dir /tmp/sendr-bundle
uv run python src/migration_cli.py verify-target --database-url "$TARGET_DB_URL" -
```

```
-upload-dir "$TARGET_UPLOAD_DIR" --quarantine-dir "$TARGET_QUARANTINE_DIR" --  
bundle-dir /tmp/sendr-bundle
```

When quarantine storage is not used, `--quarantine-dir` may be omitted and the tool will treat the upload directory as the only payload location.

Bundle Contents

The bundle contains:

- `manifest.json` with metadata, table counts, and excluded tables
- `tables/*.ndjson` with logical rows exported from SQLAlchemy models
- `files-manifest.ndjson` with checksums, scan-state-aware storage scope metadata, and file-reference metadata
- `files/` with upload payloads copied by `stored_filename`

The current implementation migrates business data and upload payloads, but intentionally excludes ephemeral runtime state such as auth sessions and verification codes. After import, users are expected to log in again.

If async malware scanning is enabled, clean payloads and quarantined payloads are both part of the migrated business state. Clean files are restored to the clean upload directory, queued or failed payloads are restored to the quarantine directory, and infected rows remain payloadless by design.

Operational Constraints

The first implementation is designed for a short maintenance window. Source writes should be frozen during the final export. Importing into a fresh target database is the default and safest mode. The tool validates missing active upload payloads, checksum mismatches, wrong storage locations for scan states, and orphaned files before export. The import path now also fails closed when the target upload or quarantine directory is not empty unless `--force` is used.

When import fails before the final commit, the importer rolls back staged database rows and removes payloads copied into the target storage directories. That guarantee is strongest in the default empty-target mode. `--force` remains an operator override for non-empty targets and should be treated as a recovery tool rather than the standard workflow.

For very large datasets, the current NDJSON exporter still uses offset-based batching and should be treated as a v1 migration path rather than a high-volume replication system.

Async Malware Scanning Deployment

The production-safe topology for async malware scanning is:

- one SendR API process
- one or more SendR scan worker processes
- a separate ClamAV daemon or sidecar-equivalent service

The API process serves user traffic. The worker runs `src/scan_worker.py` and consumes queued uploads from the database. ClamAV stays outside the backend image so definition updates are operational, not build-

time.

Both the API and worker must share:

- the same database
- the same storage backend
- the same clean upload directory and quarantine upload directory for local-disk deployments
- the same Spaces bucket configuration for object-storage deployments
- the same ClamAV connection settings

Representative backend commands are:

```
cd backend
SENDER_VIRUS_SCANNING_ENABLED=true uv run uvicorn src.app:app --host 0.0.0.0 --port 8000
SENDER_VIRUS_SCANNING_ENABLED=true uv run python src/scan_worker.py
```

The worker is intentionally separate from the API startup path. That separation keeps the web process simpler, allows worker scaling without adding web replicas, and satisfies the requirement that ClamAV updates must not require rebuilding the backend image.

With Spaces enabled, uploads are written to object storage immediately, remain blocked by `scan_status` until the worker finishes, and the worker downloads a temporary local copy before sending the file to ClamAV. Clean files stay in object storage, while infected files are deleted from storage.

Version Control and Workflow

The project was developed using Git with the repository hosted on GitHub. All source code, configuration files, and infrastructure definitions are versioned in the same repository, following a monorepo structure.

Quick Start

Backend Quick Start

```
cd backend
uv sync
uv run alembic upgrade head
SENDER_ENVIRONMENT=local uv run uvicorn src.app:app --host 0.0.0.0 --port 8000
```

Frontend Quick Start

```
cd frontend
bun install
bun run start
```

Docker Quick Start

```
docker build -t sendr-api ./backend
docker build -t sendr-frontend ./frontend
docker run -p 8000:8000 sendr-api
docker run -p 8080:8080 sendr-frontend
```